

บทและลำดับการบันทึกคลิปวิดีโอสาริระบบ DevSecOps (Demo Video Script)

โครงการ: Student Portfolio & Skill Passport (กลุ่มที่ 3) — ระบบประเมินผลงานดิจิทัล มสศ.

ผู้จัดทำและสาริต (3 คน): 1. 010 นายอภิสิทธิ์ ศรีพัฒน์ | 2. 008 นายปภักร ทองเจริญ | 3. 009 นายปวิณวัชร เหลืองอุทัย
Live URL: student-portfolio-ten-phi.vercel.app | Repository: github.com/Taeaps561/student-portfolio

ลำดับการนำเสนอตามเกณฑ์อาจารย์: ระบบจริง → เครื่องมือ → จุดอ่อนที่พบ → การแก้ไข Source Code → ผลลัพธ์หลังแก้ไข

ช่วงที่ 1: แนะนำตัว โครงการ และสถาปัตยกรรมระบบ

เวลา: 0:00 – 1:15 นาที

หน้าจอที่เปิดแสดง: หน้าแรกของระบบบน Live URL (student-portfolio-ten-phi.vercel.app) หรือ localhost:3000

"สวัสดีครับอาจารย์และเพื่อนๆ ทุกคน วันนี้กลุ่มที่ 3 จะมานำเสนอความคืบหน้าด้าน DevSecOps สำหรับโครงการ Student Portfolio & Skill Passport หรือระบบประเมินผลงานดิจิทัลและเครือข่ายวิชาชีพนักศึกษา มหาวิทยาลัยสวนดุสิตครับ"

ระบบของเราพัฒนาด้วย Next.js 16, React 19, Prisma ORM และเชื่อมต่อบ้านข้อมูล มีการใช้งานจริงบน Vercel และรองรับผู้ใช้ 3 บทบาทคือ นักศึกษา, อาจารย์, และสถานประกอบการ

ในสปรินต์นี้เราได้นำแนวคิด DevSecOps Cycle เข้ามาประยุกต์ใช้ โดยไม่เพียงแค่งานหาช่องโหว่ แต่เราได้ทำการวิเคราะห์ความเสี่ยง ลงมือแก้ไข Source Code จริง และทดสอบซ้ำตามวงจร Tool → Detect → Analyze → Fix ครับ"

ช่วงที่ 2: การใช้เครื่องมือตรวจจุดอ่อน (Detection Phase)

เวลา: 1:15 – 3:00 นาที

หน้าจอที่เปิดแสดง: โปรแกรม OWASP ZAP แสดง Alerts → Terminal รันคำสั่ง npm audit → Burp Suite Repeater

"ในขั้นตอนการตรวจจุดอ่อน (Detection) เราใช้เครื่องมือจริง 4 ชนิดกับระบบจริงของเราครับ:

- OWASP ZAP:** เมื่อรัน DAST Baseline Scan บนระบบ เราพบ Alert ระดับ Medium และ Low ถึง 5 รายการ โดยเฉพาะ Missing Security Headers ไม่มีทั้ง Content-Security-Policy และ Anti-Clickjacking (X-Frame-Options)
- npm audit (SCA):** ตรวจพบช่องโหว่ Dependencies 10 รายการ โดยมี Critical ในแพ็คเกจ next-auth และ High ใน next.js
- Burp Suite:** เราทดสอบส่งคำขอไปที่ GET /api/portfolio และ POST /api/employer/matching โดยไม่ล็อกอิน สิ่งที่พบคือ ระบบส่งข้อมูลส่วนตัวนักศึกษาออกมาหมดเลย ทั้งเบอร์โทรศัพท์ และเกรดเฉลี่ย GPA แคมโปรไฟล์ Private ก็หลุดออกมาด้วย ถือเป็นช่องโหว่ร้ายแรงด้าน Broken Access Control และละเมิด พ.ร.บ.คุ้มครองข้อมูลส่วนบุคคล (PDPA) ครับ"

ช่วงที่ 3: การวิเคราะห์และลงมือแก้ไข Source Code จริง (Remediation Phase)

เวลา: 3:00 – 5:00 นาที

หน้าจอที่เปิดแสดง: โปรแกรม VS Code เปิดไฟล์: next.config.ts, portfolio/route.ts, matching/route.ts, [...nextauth]/route.ts

"เมื่อเราทราบปัญหาแล้ว เราจึงทำการแก้ไขที่ตัวโค้ดจริง 3 จุดหลัก ไม่ใช่เพียงแค่นำภาพมารายงานครับ:

- จุดที่ 1:** ในไฟล์ next.config.ts เราเขียนฟังก์ชัน headers() เพิ่มเติมเพื่อฉีด Security Headers 7 ตัว ครอบคลุมทั้ง CSP, X-Frame-Options เป็น SAMEORIGIN ป้องกัน Clickjacking, nosniff และ HSTS
- จุดที่ 2:** ใน src/app/api/portfolio/route.ts เราแก้ปัญหา Access Control โดยเพิ่มการตรวจสอบ Session หากเป็นผู้ใช้ทั่วไป จะบังคับกรองเฉพาะโปรไฟล์สาธารณะ (isPublic: true) และเขียนฟังก์ชันทำ Data Masking ซ่อน GPA และแปลงเบอร์โทรเป็น 081-XXX-XXXX
- จุดที่ 3:** ใน src/app/api/employer/matching/route.ts เราเพิ่มการตรวจสอบสิทธิ์ Server-side RBAC ให้เฉพาะ Role EMPLOYER หรือ TEACHER เท่านั้นที่เข้าถึงข้อมูลผู้สมัครได้ หากไม่มีสิทธิ์จะถูกตอบกลับด้วย HTTP 401 Unauthorized ทันทีครับ"

🖥️ หน้าจอที่เปิดแสดง: Terminal/cURL ตรวจสอบ Header → Burp Suite Repeater ยิงคำขอซ้ำ → ไฟล์ใน evidence/before-after/

"มาดูผลลัพธ์หลังแก้ไข (After) กันครับ:

- **เมื่อยิงคำขอ curl -I ตรวจสอบ Header:** ตอนนี้ Server ส่งค่า Content-Security-Policy, X-Frame-Options: SAMEORIGIN และ X-Content-Type-Options: nosniff ครบถ้วนแล้ว และผลการสแกนซ้ำใน OWASP ZAP Alert ลดลงเหลือ **0 รายการ** ในหมวด Headers!
 - **เมื่อทดสอบผ่าน Burp Suite อีกครั้ง:** จะเห็นว่าเบอร์โทรศัพท์ถูก Mask เรียบร้อยแล้ว เกรดเฉลี่ยถูกปิดซ่อน (null) และหากพยายามเรียก API ของนายจ้างโดยไม่ล็อกอิน ระบบจะบล็อกทันทีด้วย HTTP 401 Unauthorized
- และในส่วนฐานข้อมูล เราใช้ **Prisma ORM** ซึ่งสร้าง Parameterized Queries 100% ทำให้ป้องกันการโจมตี SQL Injection ได้อย่างสมบูรณ์แบบครับ"

🖥️ หน้าจอที่เปิดแสดง: หน้า GitHub Repository แสดงไฟล์ SECURITY_PROGRESS.md, โฟลเดอร์ evidence/ และ Git Commit History

"สรุปผลการดำเนินงาน กลุ่มเราได้จัดทำเอกสารและหลักฐานครบทั้ง 4 ส่วนตามที่อาจารย์กำหนด:

1. ไฟล์ SECURITY_PROGRESS.md: สรุปรายงานความมั่นคงปลอดภัยทั้งหมดใน GitHub
2. โฟลเดอร์ evidence/: ที่มีหลักฐานของ ZAP, Burp Suite, SAST, Database และ Before/After ครบถ้วน
3. คลิปวิดีโอสาธิตนี้
4. ประวัติ Git Commit บน GitHub: ที่ยืนยันว่ามีการแก้ไขโค้ดจริง

ในสปรินต์ถัดไป เราพร้อมที่จะนำ GitHub Actions ของเรามาต่อยอดเป็น CI/CD Security Pipeline เต็มรูปแบบครับ กลุ่มที่ 3 ขอจบการนำเสนอเพียงเท่านี้ขอบคุณครับ"

💡 คำแนะนำสำหรับการบันทึกวิดีโอสาธิต (Video Recording Checklist)

- **โปรแกรมบันทึกหน้าจอ:** แนะนำให้ใช้ OBS Studio หรือ ShareX โดยตั้งค่าความละเอียด Full HD (1080p) และเปิดกล้องเว็บแคมมุมเล็กๆ เพื่อความเป็นมืออาชีพ
- **การเตรียมหน้าตาหน้าจอ:** ปิดแท็บเว็บจริงบนเบราว์เซอร์, VS Code (เปิดไฟล์ next.config.ts และ route.ts), โปรแกรม OWASP ZAP, และ Burp Suite เตรียมไว้ล่วงหน้า จะช่วยให้สลับหน้าจอได้อย่างราบรื่น
- **การอัปโหลดและส่งงาน:** อัปโหลดคลิปขึ้น YouTube (ตั้งค่าเป็น Unlisted หรือ Public) หรือ Google Drive (เปิดสิทธิ์ Anyone with the link can view) แล้วนำลิงก์มาแนบส่งในระบบส่งงานของรายวิชา